Resumen

En el Proyecto Final I desarrollé los servicios de contactos, tareas y citas para una aplicación móvil. Mi enfoque de pruebas unitarias consistió en verificar las reglas de validación de cada objeto y el comportamiento de las operaciones disponibles en cada servicio. Utilicé JUnit para comprobar resultados esperados y para confirmar que ciertas entradas inválidas generaran excepciones. Este enfoque permitió convertir los requisitos en verificaciones concretas, aunque la cobertura total debe medirse con una herramienta específica.

En el servicio de contactos, ContactTest.java, líneas 26–40, comprueba que el identificador no sea nulo ni supere diez caracteres. Las líneas 66–80 verifican que el teléfono tenga el formato esperado y no sea nulo. Para las operaciones del servicio, ContactServiceTest.java, líneas 32–49, comprueba el rechazo de identificadores duplicados mediante assertThrows. Además, las líneas 66–77 verifican la actualización del nombre y las líneas 105–116 la actualización de la dirección. Estas pruebas corresponden a los requisitos de validación, unicidad y actualización de la información de contacto.

En el servicio de tareas, TaskTest.java, líneas 22–44, verifica que el identificador no sea nulo ni exceda diez caracteres; las líneas 58–68 comprueban el límite de veinte caracteres del nombre y las líneas 82–92 el límite de cincuenta caracteres de la descripción. TaskServiceTest.java, líneas 123–149, verifica que una operación de actualización cambie tanto el nombre como la descripción. Las líneas 69–75 también prueban el rechazo de la eliminación de una tarea inexistente. Así se cubren validaciones del objeto y comportamientos del servicio.

En el servicio de citas, AppointmentTest.java, líneas 43–57, verifica que la fecha no sea nula ni esté en el pasado. Las líneas 59–76 comprueban la descripción y su límite de longitud, y las líneas 91–105 verifican que se pueda actualizar la fecha. En AppointmentServiceTest.java, líneas 37–58, se prueba el rechazo de citas duplicadas; las líneas 76–88 comprueban el rechazo de la eliminación de una cita inexistente y de la adición de una cita nula. A diferencia del servicio de tareas, el servicio de citas implementado no incluye un método de actualización en la clase de servicio.

Las pruebas son más útiles cuando cubren tanto los casos válidos como los inválidos y verifican el resultado esperado. Por ejemplo, assertEquals comprueba que el valor real coincida con el esperado, mientras que assertThrows verifica que una operación produzca la excepción prevista (JUnit Team, s. f.). El archivo ZIP entregado contiene código fuente y pruebas, pero no incluye un informe de cobertura; por ello, no es posible afirmar un porcentaje real a partir de estos archivos solamente. Para completar esta parte de la rúbrica, se debe ejecutar una herramienta de cobertura, como JaCoCo, y reportar el porcentaje de líneas e instrucciones cubiertas que aparezca en el informe generado.

Mi experiencia al escribir pruebas me enseñó a distinguir entre solidez técnica y eficiencia. Una prueba es técnicamente útil cuando verifica una condición específica y falla si el comportamiento es incorrecto. Por ejemplo, ContactServiceTest.java, líneas 46–48, verifica que no se agregue un contacto duplicado. TaskServiceTest.java, líneas 140–148, comprueba por separado los valores actualizados del nombre y la descripción. La eficiencia se favorece al organizar las pruebas por comportamiento y usar @BeforeEach para preparar un servicio nuevo antes de cada prueba, como se observa en ContactServiceTest.java, líneas 12–15, y

AppointmentServiceTest.java, líneas 14–17. Esta preparación reduce la dependencia entre pruebas y facilita identificar la causa de un fallo.

Reflexión

Técnicas de prueba

La técnica principal empleada fue la prueba unitaria: evalué objetos y métodos de forma individual para comprobar reglas y resultados. También utilicé pruebas positivas, como la creación válida de una tarea en TaskTest.java, líneas 8–20, y pruebas negativas, como el rechazo de una fecha pasada en AppointmentTest.java, líneas 50–57. Las pruebas de límites se reflejan en la comprobación de identificadores y longitudes máximas. Para fortalecerlas aún más, convendría probar explícitamente valores justo en el límite permitido y justo por encima de él, porque los errores suelen aparecer en esas fronteras.

No se incluyeron pruebas de integración, de sistema, de aceptación ni de rendimiento como conjuntos independientes. Las pruebas de integración verifican que varios componentes colaboren correctamente; las de sistema evalúan la aplicación completa; las de aceptación comprueban si el producto satisface las necesidades del cliente; y las de rendimiento miden aspectos como el tiempo de respuesta o el comportamiento bajo carga. Estas técnicas se complementan: las pruebas unitarias localizan errores en componentes pequeños, mientras que las pruebas de nivel superior detectan problemas de interacción y funcionamiento global (Pressman & Maxim, 2020). Por ejemplo, si los servicios se conectaran posteriormente a una base de datos, serían necesarias pruebas de integración para verificar el almacenamiento y la recuperación de registros.

Mentalidad de evaluación

Al evaluar el código, procuré no asumir que compilar equivalía a funcionar correctamente. Revisé escenarios como identificadores duplicados, valores nulos, longitudes excesivas y fechas pasadas. Estas condiciones son importantes porque una entrada inválida puede causar fallos o datos inconsistentes. También es importante considerar las interrelaciones: una actualización puede depender de que el registro exista y de que los nuevos valores respeten las reglas de validación. En TaskService.java, por ejemplo, el método updateTask obtiene la tarea y después llama a los métodos de validación del objeto; por eso, las pruebas deben comprobar tanto la existencia de la tarea como la validez de los nuevos datos.

El sesgo puede aparecer cuando un desarrollador prueba su propio código y se concentra en demostrar que funciona, en vez de intentar descubrir cómo podría fallar. Para limitarlo, conviene comparar cada prueba con los requisitos, diseñar casos negativos y revisar el código con otra persona cuando sea posible. Por ejemplo, probar únicamente una fecha futura válida no demostraría que el servicio rechaza una fecha pasada; para ello se necesita un caso específico, como el que aparece en AppointmentTest.java, líneas 50–57. También debo reconocer los límites de mis pruebas: una prueba exitosa demuestra el comportamiento en el escenario evaluado, no la ausencia absoluta de defectos.

La disciplina es esencial para evitar deuda técnica y mantener la calidad del software. Saltarse pruebas puede ahorrar tiempo al principio, pero dificulta detectar la causa de defectos después de introducir nuevos cambios. Para reducir ese riesgo, mantendré pruebas automatizadas, ejecutaré el conjunto de pruebas después de cada modificación importante, revisaré los casos límite y documentaré las decisiones relevantes. También procuraré corregir la causa de los defectos en lugar de añadir soluciones temporales. Las pruebas

deben mantenerse junto con el código, ya que los requisitos y las implementaciones pueden cambiar con el tiempo.

Referencias

- JUnit Team. (s. f.). *JUnit 5 user guide*. https://docs.junit.org/6.1.3/overview.html
- Pressman, R. S., & Maxim, B. R. (2020). *Software engineering: A practitioner's approach* (9th ed.). McGraw Hill.